

Estruturas de Repetição no Arduino (FOR e WHILE)

Até agora, nos projetos anteriores, o Arduino:

Reagia a eventos

Exemplo: apertar botão → acender LED

Agora vamos evoluir

O Arduino não precisa mais só reagir, ele pode executar ações repetidas automaticamente.

CONEXÃO COM PORTUGOL - IMPORTANTÍSSIMO

- **No Portugol (VisualG)**

Estrutura PARA

```
para i de 1 até 5 faça  
  escreva("Olá")  
fimpara
```

Significa:

Repita 5 vezes

Estrutura ENQUANTO

```
enquanto contador < 5 faça  
  escreva(contador)  
  contador ← contador + 1  
fimenquanto
```

Significa:

Repete enquanto a condição for verdadeira

TRADUÇÃO PARA ARDUINO

Portugol	Arduino
para	for
enquanto	while
contador ← contador + 1	contador++

ATIVIDADE 1 — Piscar LED 5 vezes (FOR)

Objetivo: Fazer o LED piscar automaticamente **5 vezes**

Lógica (como no Portugol)

```
para i de 1 até 5 faça
  liga LED
  espera
  desliga LED
  espera
fimpara
```

Código Arduino

```
int led = 12;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {

  // Estrutura de repetição FOR

  // Ela vai repetir o bloco de código 5 vezes
```

```
// int i = 0 -> começa o contador em 0
// i < 5    -> repete enquanto i for menor que 5
// i++      -> soma 1 ao contador a cada repetição
for (int i = 0; i < 5; i++) {

    // Liga o LED
    // HIGH significa nível alto, ou seja, envia energia para o pino
    digitalWrite(led, HIGH);
    delay(500); // Espera 500 milissegundos (meio segundo)

    // Desliga o LED
    // LOW significa nível baixo, ou seja, corta a energia do pino
    digitalWrite(led, LOW);
    delay(500); // Espera 500 milissegundos (meio segundo)
}

/ Depois de piscar 5 vezes, o Arduino espera 2 segundos
// antes de começar tudo novamente
delay(2000);
}
```

Explicação

```
for (int i = 0; i < 5; i++)
```

Tradução:

“Repita 5 vezes”

O for é usado quando eu já sei quantas vezes quero repetir.

ATIVIDADE 2 — Piscar LED enquanto botão estiver pressionado (WHILE)

Objetivo: Enquanto o botão estiver pressionado → LED pisca

Lógica em Portugol

```
enquanto botão pressionado faça  
    pisca LED  
fimenquanto
```

Código Arduino

```
int botao = 8; // Declara o pino onde o botão está conectado
```

```
int led = 12; // Declara o pino onde o LED está conectado
```

```
void setup() {
```

```
    pinMode(botao, INPUT); // Define o pino do botão como entrada
```

```
    pinMode(led, OUTPUT); // Define o pino do LED como saída
```

```
}
```

```
void loop() {
```

```
    // Estrutura WHILE
```

```
    // Enquanto o botão estiver pressionado (HIGH)
```

```
    // o código dentro do while será executado
```

```
    while (digitalRead(botao) == HIGH) {
```

```
        // Liga o LED
```

```
digitalWrite(led, HIGH);
```

```
delay(300); // Espera 300 milissegundos
```

```
// Desliga o LED
```

```
digitalWrite(led, LOW);
```

```
delay(300); // Espera mais 300 milissegundos
```

```
}
```

```
}
```

Explicação

while (condição)

Significa:

Enquanto isso for verdadeiro, continue

O while depende de uma condição, ele não sabe quando vai parar.

ATIVIDADE 3 — Contador com WHILE (Serial)

Objetivo: Mostrar contagem automática no Serial

Lógica em Portugol

contador $\leftarrow$ 0

enquanto contador $<$ 5 faça

 escreva(contador)

 contador $\leftarrow$ contador + 1

fimenquanto

Código Arduino

```
// Declara uma variável chamada contador
// Ela vai armazenar números (0, 1, 2, 3, 4...)
int contador = 0;

void setup() {

  // Inicia a comunicação com o computador
  // Isso permite mostrar informações no Monitor Serial
  Serial.begin(9600);
}

void loop() {

  // Sempre que o loop reiniciar, o contador volta para 0
  // Isso faz a contagem começar do início novamente
  contador = 0;

  // Estrutura WHILE
  // Enquanto o contador for menor que 5, o código dentro será executado
  while (contador < 5) {

    // Mostra o valor atual do contador no Monitor Serial
    Serial.println(contador);

    // Incrementa o contador (soma +1)
    // Ex: 0 → 1 → 2 → 3 → 4
    contador++;

    // Espera 500 milissegundos (meio segundo)
    delay(500);
  }

  // Quando o contador chega a 5, o WHILE para
  // Então o Arduino espera 2 segundos antes de recomeçar tudo
  delay(2000);
}
```

O que você aprende:

- variável como memória
- incremento
- controle de parada

ATIVIDADE 4 — Integração (Botão + FOR)

Objetivo: Ao pressionar o botão → LED pisca 3 vezes

Lógica em Portugol

```
se botão pressionado então  
  para i de 1 até 3 faça  
    pisca LED  
  fimpara  
fimse
```

Código Arduino

```
int botao = 8; // Declara o pino onde o botão está conectado  
  
int led = 12; // Declara o pino onde o LED está conectado  
  
int estadoAtual = 0; // Variável que guarda o estado atual do botão (HIGH ou LOW)  
  
int estadoAnterior = 0; // Variável que guarda o estado anterior do botão
```

```
void setup() {
```

```
  pinMode(botao, INPUT); // Define o pino do botão como entrada
```

```
  pinMode(led, OUTPUT); // Define o pino do LED como saída
```

```
}
```

```
void loop() {
```

```
// Lê o estado atual do botão HIGH → pressionado, LOW → solto  
estadoAtual = digitalRead(botao);
```

```
// Detecta um clique (transição de LOW para HIGH)
```

```
// Ou seja:
```

```
// antes estava solto (LOW) e agora foi pressionado (HIGH)
```

```
if (estadoAtual == HIGH && estadoAnterior == LOW) {
```

```
    // Estrutura FOR
```

```
    // Vai repetir o código 3 vezes
```

```
    // i começa em 0
```

```
    // repete enquanto i < 3
```

```
    // i++ soma +1 a cada repetição
```

```
    for (int i = 0; i < 3; i++) {
```

```
        digitalWrite(led, HIGH); // Liga o LED
```

```
        delay(300); // Espera 300 milissegundos
```

```
        digitalWrite(led, LOW); // Desliga o LED
```

```
        delay(300); // Espera mais 300 milissegundos
```

```
    }
```

```
}
```

```
// Atualiza o estado anterior
```

```
// Isso é essencial para detectar o próximo clique
```

```
estadoAnterior = estadoAtual;
```


}

Comparação final

Estrutura Quando usar

for Quando sabe quantas vezes repetir

while Quando depende de uma condição

Antes o Arduino só reagia, agora ele executa tarefas automaticamente, repetindo ações de forma controlada.